Firing Rules and Semiring: A Comparative Study of Why-Not Provenance

Lee Zhi Yi (A0258639L), Phan Dinh Hung (A0255791U), and Himanshu Maithani (A0314584B)

National University of Singapore

Abstract. We abstract a why-not provenance system as a five-stage pipeline consisting of query normalization, candidate derivation space construction, derivation evaluation, explanation extraction, and explanation presentation. Given a query Q, database instance D, and provenance question $WHYNOT_Q(t)$, the system identifies derivations relevant to t, evaluates why each derivation fails under D, and transforms these failures into an interpretable explanation artifact. Under this abstraction, different why-not provenance approaches can be understood as differing primarily in how they represent derivations, how exhaustively they explore the derivation space, and how they encode or summarize explanation outputs. Our firing-rules approach instantiates this framework by lowering queries into rule form, enumerating relevant bindings, evaluating grounded goals at the derivation level, and returning failed derivations together with a graph-structured explanation. Our semiring-based approach instantiates the same pipeline differently: queries are parsed into a relational operator tree, each base tuple is annotated with a semiring token, semiring operations are propagated through evaluation, and a missing tuple is explained by tracing where its annotation was annihilated to zero through the operator tree.

Keywords: why-not provenance · semiring · firing rules · Datalog · query explanation

1 Introduction

Why-not provenance seeks to explain why an expected tuple is absent from a query result. At a high level, such approaches can be understood as operating over the space of derivations that could have produced the target tuple, but fail under the current database instance. Despite differences in formalism, most methods can be described through a common workflow: the query is first normalized into an internal representation, relevant candidate derivations are then constructed, each derivation is evaluated to identify the conditions under which it fails, and the resulting failures are transformed into an explanation artifact such as failed derivations, missing witnesses, symbolic expressions, or summary patterns. This abstraction is useful because it provides a uniform lens for comparing rule-based, graph-based, algebraic, and summarized why-not provenance techniques.

In this paper, we instantiate this general view through two approaches. First, a firing-rules approach that translates queries into Datalog form, enumerates candidate derivations, evaluates each goal individually, and constructs a graph-structured why-not explanation from the set of failed derivations. Second, a semiring-based approach that annotates base tuples with provenance tokens, evaluates relational operators using semiring arithmetic, and identifies the root cause of a missing answer by tracing the path through the operator tree along which the annotation was driven to zero.

2 Background

Why-not provenance studies why an expected tuple is absent from a query result. More generally, provenance concerns the relationship between query outputs and the input data, and can be expressed through multiple representations, including rule-based derivations, provenance graphs, and algebraic annotations. In the rule-based view, a query is understood through the derivations that could produce a tuple, and a missing answer arises when all relevant derivations fail. In the algebraic view, provenance is represented symbolically through semiring expressions that capture how outputs depend on inputs. These perspectives share a common goal: to characterize the conditions under which a tuple would or would not appear in the result.

Provenance analysis relies on a few core concepts: base relations stored in the database, derived query outputs, candidate witnesses or derivations for a target tuple, and evaluation conditions such as positive matches, negated conditions, and comparisons. A why-not provenance method therefore needs some way to represent possible explanations for absence, evaluate them against the database instance, and return an explanation artifact. This general foundation provides the conceptual basis for both rule-based and semiring-based provenance techniques, as well as later extensions for summarization and scalable explanation.

3 Related Work

Research on why-not provenance has developed along several complementary lines.

A major direction is rule- and graph-based explanation, where missing answers are explained by examining the derivations that could have produced a target tuple and identifying where those derivations fail. Provenance games provide one such unified treatment for first-order queries, capturing both successful and failed derivations within a game-theoretic model [2]. Building on this direction, Lee et al. proposed a SQL-middleware approach that instruments Datalog rules through firing rules to compute why and why-not explanations for first-order queries with negation, focusing on practical execution over database systems [3]. The PUG system further operationalized this line of work as a practical framework for why and why-not provenance, emphasizing graph-based explanation and usability [4].

A second direction comes from algebraic provenance. Semiring-based provenance offers a general framework for tracking how outputs depend on inputs across multiple query semantics, including why-provenance, bag semantics, probabilistic databases, and incomplete databases [1]. This framework is highly expressive and compositional, but its primary strength lies in representing successful dependency structure; direct explanation of missing answers typically requires additional mechanisms beyond standard positive provenance representations.

A third line of work addresses scalability and interpretability. Since why-not provenance can become extremely large under all-derivations semantics, Lee et al. propose pattern-based approximate summaries balancing completeness, informativeness, and conciseness through sampling and top-k summary selection [5].

Our work is most closely aligned with the rule-based tradition but focuses specifically on executable architectures for why-not provenance over a Datalog + SQL subset, making failed derivations, goals, and graph-structured explanations directly available in an implementation-oriented setting.

4 Methodology

We implemented two distinct approaches to why-not provenance, both evaluated on a TPC-C benchmark database consisting of `items`, `warehouses`, and `stocks` relations. Both approaches follow the five-stage pipeline described in the abstract but differ substantially in their internal representation, evaluation strategy, and explanation mechanism.

4.1 Firing-Rules Approach

The firing-rules approach explains why-not provenance by translating a query into Datalog form, enumerating the candidate derivations that could have produced a target tuple, evaluating each goal of each derivation individually, and collecting the failed derivations as evidence for absence. The central idea is that a tuple is missing because every possible relevant derivation fails, and the explanation identifies which goals within each derivation are responsible for that failure.

Step 1: Query Normalization. The query is first expressed as a set of Datalog rules. Each rule has a head predicate and a body consisting of positive goals (EDB lookups), negated goals (`NOT EXISTS` checks), and comparison predicates. For example:

$$r_1 : Q(X, Y) :- \text{Train}(X, Z),\ \text{Train}(Z, Y),\ \neg\, \text{Train}(X, Y).$$

This normalized form is the basis for all subsequent derivation enumeration and evaluation.

Step 2: Provenance Question. The user specifies a why-not question of the form $WHYNOT\ Q(a, b)$, identifying the target predicate and the missing tuple. Only derivations whose head matches this pattern need to be considered, avoiding evaluation of the full derivation space.

Table 1. Example firing table for *WHYNOT* $Q(s, n)$ under rule r_1.

X	Y	Z	g_1^{ok}	g_2^{ok}	g_3^{ok}	status
s	n	c	false	false	true	false
s	n	w	false	false	true	false
s	n	s	false	false	true	false
s	n	n	false	false	true	false

Step 3: Domain and Binding Enumeration. For each variable in a rule, a domain is constructed from the active domain of the database. Candidate derivations are formed by enumerating all variable bindings consistent with the target tuple pattern. For *WHYNOT* $Q(s, n)$ with Z ranging over $\{n, s, c, w\}$, the candidates are (X=s, Y=n, Z=c), (X=s, Y=n, Z=w), (X=s, Y=n, Z=s), and (X=s, Y=n, Z=n).

Step 4: Per-Goal Evaluation and Firing Tables. For each candidate derivation, every goal in the rule body is evaluated independently against the database. The results are recorded in a *firing relation* $F_{r_1}(X, Y, Z, g_1^{ok}, g_2^{ok}, g_3^{ok}, \mathit{status})$, where each g_i^{ok} records whether goal i succeeded and $\mathit{status} = g_1^{ok} \wedge g_2^{ok} \wedge \cdots \wedge g_n^{ok}$. Table 1 shows an example firing table for *WHYNOT* $Q(s, n)$.

Step 5: Explanation Graph Construction. The failed derivations are assembled into a provenance graph with three node types: *tuple nodes* representing present or absent tuples, *rule nodes* representing specific derivations, and *goal nodes* representing individual goals within a derivation. For a why-not explanation, the graph connects the missing tuple node to its failed rule nodes, and each failed rule node to its failed goal nodes.

Step 6: Output. The system returns the set of failed derivations together with the graph-structured explanation. For the example above, the explanation indicates that $Q(s, n)$ is absent because all four candidate derivations fail, each blocked by the absence of $\mathrm{Train}(s, Z)$ and $\mathrm{Train}(Z, n)$ for any intermediate node Z.

4.2 Semiring-Based Approach: Theory

Our semiring-based approach is grounded in the K-relations framework of Green et al. [1]. A *semiring* is a tuple $\mathbb{K} = (K, \oplus, \otimes, \mathbf{0}, \mathbf{1})$ where $(K, \oplus, \mathbf{0})$ is a commutative monoid, $(K, \otimes, \mathbf{1})$ is a monoid, $\otimes$ distributes over $\oplus$, and $\mathbf{0}$ is an annihilator: $\mathbf{0} \otimes a = \mathbf{0}$ for all a. A *K-relation* $R : \mathrm{Tup} \to K$ assigns an annotation to every tuple; tuples with annotation $\mathbf{0}$ are absent from the relation.

The standard relational operators lift to K-relations as follows:

$$\big(\sigma_\theta(R)\big)(t) = \begin{cases} R(t) & \text{if } \theta(t) \\ \mathbf{0} & \text{otherwise} \end{cases} \tag{1}$$

$$\big(\pi_S(R)\big)(t) = \bigoplus_{t' :\, \pi_S(t') = t} R(t') \tag{2}$$

$$(R \bowtie S)(t) = R(\pi_A(t)) \otimes S(\pi_B(t)) \tag{3}$$

$$(R \cup S)(t) = R(t) \oplus S(t) \tag{4}$$

The key insight for why-not is that a missing tuple t satisfies $Q_{\mathbb{K}}(D)(t) = \mathbf{0}$. Explaining its absence is equivalent to tracing the operator tree to find the node where the annotation was annihilated.

We implement four semirings forming a strict information order $\mathbb{B} \sqsubseteq \mathbb{N} \sqsubseteq \mathbb{W} \sqsubseteq \mathbb{H}$:

Boolean $\mathbb{B} = (\{0,1\}, \vee, \wedge, 0, 1)$. Each base tuple gets annotation 1. The output is 1 (present) or 0 (absent). This confirms a tuple is missing but gives no reason.

Bag $\mathbb{N} = (\mathbb{N}, +, \times, 0, 1)$. Each base tuple gets annotation 1. The output annotation counts distinct derivation paths. A zero multiplicity signals absence.

Why-provenance $\mathbb{W} = (2^{2^T}, \cup, \bowtie, \emptyset, \{\emptyset\})$. Here $A \bowtie B = \{a \cup b \mid a \in A, b \in B\}$ and T is the set of base-tuple tokens. An annotation is a set of *witnesses*, each witness being a set of base-tuple tokens co-occurring in one derivation. A base tuple t_i with token τ_i receives annotation $\{\{\tau_i\}\}$. The empty set $\emptyset$ (no witnesses) characterises a missing tuple, and its witness set points directly to which base tuples are involved in each failed derivation path.

How-provenance $\mathbb{H} = (\mathbb{N}[X], +, \times, 0, 1)$. Annotations are polynomials over base-tuple variables. Each base tuple t_i receives annotation x_i. The polynomial of a present tuple encodes every derivation path (monomials) with its multiplicity (coefficients). $\mathbb{H}$ is the most expressive semiring and subsumes all others via semiring homomorphisms [1].

Given the evaluation trace, three root causes arise from the operator semantics above:

1. **Source missing**: no base scan produces t; annotation is $\mathbf{0}$ at the leaf.
2. **Predicate failure**: a matching tuple exists in the child relation but $\theta(t)$ is false, driving annotation to $\mathbf{0}$ via (1).
3. **Join partner missing**: one join side has a match but the other has no partner, annihilating the product via (3).

A fourth informational case, **column hidden**, occurs when the queried column was projected away by π.

4.3 Firing-Rules Implementation

The firing-rules system is a modular Python implementation that accepts mixed input specifications and returns a JSON explanation. The codebase is organized as follows: `why_not_provenance.py` is a CLI wrapper that reads from stdin or a file argument and prints JSON; `provenance/api.py` exposes a public `explain_why_not(query_text)` entry point; `provenance/input_parser.py` handles mixed-input normalization, optional PostgreSQL loading, and SQL execution; `provenance/sql_rule_builder.py` lowers SQL blocks to Datalog rules; `provenance/engine.py` performs binding generation, goal evaluation, and explanation construction; and and `provenance/helpers.py` together with `provenance/models/` supply shared parsing utilities and typed dataclasses.

Internal representation. The parser produces a `Program` object holding the EDB (base facts by predicate), declared schemas, explicit Datalog rules, SQL-lowered rules, the provenance question, and optional SQL execution payloads. Each rule-body goal is typed as `pos` (positive atom), `neg` (negated atom), or `cmp` (comparison). Values are normalized to strings at ingestion; numeric comparison operators attempt coercion when both operands parse as numbers. Variables follow an ALL_CAPS convention (`[A-Z_][A-Z0-9_]*`) to avoid treating string literals such as `Singapore` as variables.

Input parsing and normalization. The parser scans input blocks in order, recognizing `SCHEMA` declarations, EDB facts, Datalog rules, SQL blocks, the provenance question, and an optional connection directive. When a PostgreSQL connection is available — resolved from environment variables `WHY_NOT_DB_CONNECTION`, `DATABASE_URL`, or `POSTGRES_CONNECTION_STRING` (with optional `.env` loading) — the system fetches all referenced base tables into the in-memory EDB and executes any SQL blocks, retaining raw results in the output payload. Table loading and SQL execution can be parallelized via `WHY_NOT_MAX_WORKERS`.

SQL-to-rule lowering. Each SQL block is translated into one Datalog-like rule under a restricted fragment supporting `SELECT-FROM-JOIN-WHERE`, `AND` conjunctions, equality and inequality comparisons ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$), and a limited `NOT EXISTS` pattern. Table references become positive atom goals with synthetic variables (e.g., `S_W_ID`, `I_I_PRICE`); join and where predicates become comparison goals, or negated atom goals for supported `NOT EXISTS` forms; and `SELECT` expressions become head terms. With a single SQL block the generated head predicate matches the question predicate; with multiple blocks, heads are named `QSQL1`, `QSQL2`, and so on.

WHY-NOT evaluation. Given the target tuple $Q(t_1, \ldots, t_n)$, the engine proceeds in three phases.

1. **Head materialization.** Initialize the relation store from the EDB, enumerate candidate bindings for each rule, and record successful head tuples.

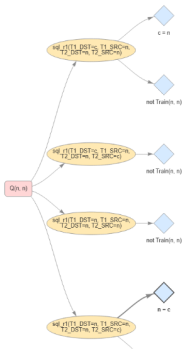

Fig. 1. Explanation graph for *WHYNOT* $Q(n, n)$. Pink square: missing tuple node. Ovals: failed rule instances, labelled by variable binding. Blue diamonds: failing goals (comparison failure or negated atom).

2. **Target-rule filtering.** Retain only rules whose head predicate matches the target; constrain head variables to target values where applicable.
3. **Failed-derivation extraction.** For each retained rule, enumerate bindings using a join-driven strategy: positive goals are unified against relation rows first, then remaining unbound variables are expanded over their active-domain values. Each binding is grounded and every body goal is evaluated independently (positive atom membership, negated atom absence, or comparison). Bindings whose grounded head equals the target tuple and whose overall status is false constitute the failed derivations.

The join-driven strategy avoids constructing the full Cartesian product of variable domains in most cases, but worst-case combinatorial growth can still occur for rules with many unbound variables after positive goals are exhausted.

Output. The JSON output contains `mode`, `target`, `failed_derivation_count`, `failed_derivations` (one record per failed binding with per-goal status), `explanation_graph`, `query_results`, and `message`. Each failed-derivation record follows the structure introduced in Section 4.1: a `rule_id`, the grounded `binding`, a `goal_results` list with the outcome of each goal, and a boolean `status`. Figure 1 shows the explanation graph for *WHYNOT* $Q(n, n)$: the missing tuple node (pink square) connects to four `sql_r1` rule instances (ovals), each linked to its failing goal nodes (blue diamonds), which are either comparison failures or negated-atom checks.

Scope and limitations. Only the WHY-NOT output path is implemented; the WHY (why-present) path is parsed but not evaluated. SQL support is restricted to the fragment above: no aggregates, no `OR`, and no complex nesting. The implementation is intentionally scoped to this subset, which covers all four operator categories (SELECT, PROJECT, JOIN, UNION) used in the benchmark.

4.4 Semiring Implementation

The semiring system implements the five pipeline stages as follows.

Parser. A SQL string is parsed into a logical operator tree of five node types (`Scan`, `Select`, `Project`, `Join`, `Union`), built bottom-up following standard relational algebra precedence.

Annotator. The Annotator fetches all rows from each base table and assigns a unique provenance token to every tuple. If the ProvSQL extension [6] is available, it uses the native UUID gate; otherwise tokens are constructed from the primary key (e.g. `items_3`, `stocks_301_1`).

Evaluator. The Evaluator traverses the operator tree recursively, applying semiring operations at each node per equations (1)–(4). It returns an `EvaluationTrace`: a mirrored tree holding the intermediate K-relation at every node. All four semirings share this dispatch; only the operations differ. `WhyProvenance` and `HowProvenance` use Python `frozenset` and `dict` objects for structural annotation operations without database round-trips.

WhyNotEngine. Given a missing tuple, the engine walks the `EvaluationTrace` and applies the zero-detection logic to identify the root cause and blocking operator. The `EvaluationTrace` is re-used from the evaluation pass, so diagnostic cost is $O(h)$ in tree height rather than requiring a full re-execution.

Explainer. The Explainer formats the diagnosis as a structured report including the cause label, a plain-English suggestion, and the annotation under each of the four semirings — illustrating the escalating expressiveness of the information order.

5 Performance Evaluation

5.1 Experimental Setup

Both systems were evaluated on the same TPC-C PostgreSQL database (`tpcc_db`) using a shared suite of 16 queries spanning four operator categories: SELECT (Q01–Q04), PROJECT (Q05–Q06), 2-way JOIN (Q07–Q10), and UNION with 2 or 3 branches (Q15–Q20), including unions of joins. The schema comprises `items` (483 rows), `warehouses` (≈50 rows), and `stocks` (≈21 000 rows). All timing figures are the median over five runs (milliseconds). Space is measured as the total serialised character length of all annotations across the output K-relation (semiring) or the JSON explanation for one missing tuple (firing rules).

Table 2. Execution time (ms, median over 5 runs). FR = Firing Rules.

Query	Operator	FR	Boolean	Bag	Why	How	FR/How
Q01	SELECT	87.3	5.8	5.1	5.9	5.5	15.8×
Q02	SELECT	50.5	3.2	3.2	3.4	3.4	14.9×
Q03	SELECT	50.2	3.7	3.7	4.0	3.8	13.2×
Q04	SELECT	719.2	247.0	244.7	375.7	303.7	2.4×
Q05	PROJECT	129.7	5.6	5.5	5.8	5.7	22.7×
Q06	PROJECT	160.1	3.1	3.2	3.4	3.2	50.0×
Q07	JOIN	16449.6	1567.5	1540.5	1743.3	1743.7	9.4×
Q08	JOIN	16415.2	1507.7	1492.3	1689.5	1668.2	9.8×
Q09	JOIN	31322.0	2847.3	2822.9	3003.0	2924.9	10.7×
Q10	JOIN	16475.5	1481.1	1463.8	1595.7	1633.4	10.1×
Q15	UNION (2)	141.8	10.1	10.4	11.4	10.9	13.0×
Q16	UNION (2)	71.7	5.4	5.6	5.9	5.7	12.6×
Q17	UNION (3)	82.7	8.7	9.0	8.9	9.1	9.1×
Q18	UNION+JOIN (2)	32806.7	3023.8	2993.7	3708.9	3591.7	9.1×
Q19	UNION+JOIN (2)	60847.4	5201.9	5217.5	5750.8	5708.2	10.7×
Q20	UNION+JOIN (3)	47390.6	4470.1	4487.6	5364.5	5204.7	9.1×

5.2 Time Analysis

Table 2 gives the full timing results and Figures 2–3 visualise the comparison.

The semiring approach is consistently faster across all query types, with a speedup ranging from 2.4× (Q04) to 50× (Q06). JOIN and UNION-of-JOIN queries converge on a stable 9–11× advantage. The semiring system delegates execution to PostgreSQL, which uses optimised join algorithms and buffer caching, whereas the firing-rules system loads full tables into Python memory and evaluates rules by nested-loop iteration.

Within the semiring system, Boolean and Bag are marginally faster than Why and How by ≈10–15% on JOIN queries, reflecting annotation construction overhead that is small relative to the database fetch cost.

Q04 anomaly. Q04 returns 21 302 output tuples (stocks with `s_qty`>500). At this output size the semiring system spends significant time materialising the K-relation in Python, narrowing the gap to 2.4×.

Q09. Despite producing only 614 output tuples, Q09 (stocks ⋈ warehouses) forces a full scan of ≈21 000 stock rows per warehouse, making it the most expensive 2-way join on both sides (FR: ≈31 s; semiring: ≈2.9 s).

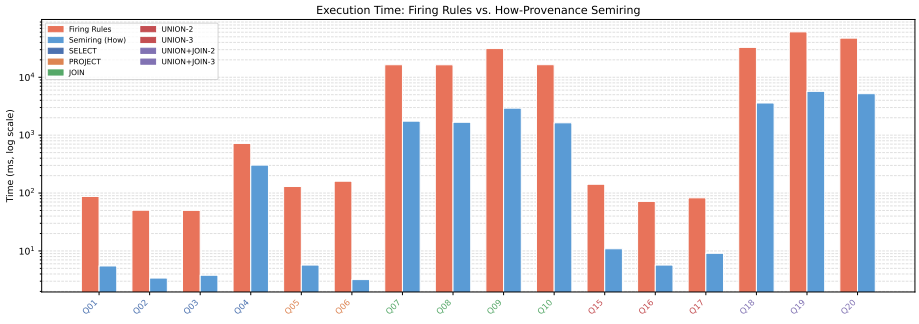

Fig. 2. Execution time (log scale): Firing Rules vs. How-Provenance. Tick-label colour indicates operator category.

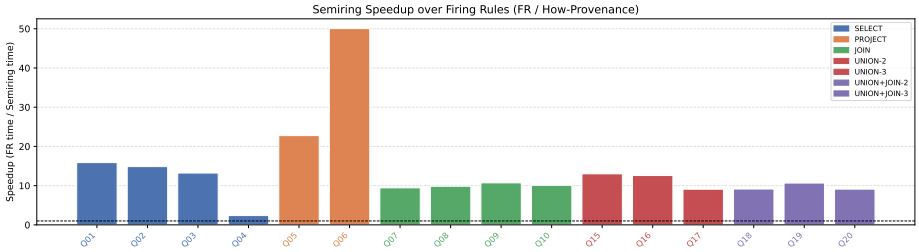

Fig. 3. Semiring speedup over Firing Rules (FR / How time).

5.3 Space Analysis

Table 3 and Figure 4 reveal a fundamental design contrast. **Firing-rules space is bounded and stable**: because the system explains one missing tuple at a time, output size depends only on rule count and body depth, not on result set size. The FR JSON ranges from 942 to 3 904 chars across all 16 queries. The sole outlier is Q05 (1 035 532 chars), which enumerates one failed derivation per non-Singapore warehouse to explain the projected value's absence.

Semiring space grows with result size and join depth. Boolean and Bag annotations are trivially small. Why and How annotations grow significantly because every output tuple carries its full provenance: each join multiplies token references and each UNION branch adds witnesses. For Q07, Q18, and Q20 the Why/How cumulative size reaches 1–1.6 MB.

Q05 PROJECT contrast. The semiring system produces one output tuple `(Singapore)` with a Why annotation of 164 chars (one witness per Singapore warehouse). The firing-rules system returns 963 failed derivations totalling 1 035 532 chars. This exposes the key design difference: the semiring system annotates *what is present*, while the firing-rules system explains *why a specific value is absent*.

Table 3. Annotation / explanation size (serialised characters). FR = JSON explanation for one missing tuple. Semiring columns = cumulative annotation size over the full output relation.

Query	Operator	FR JSON	Boolean	Bag	Why	How
Q01	SELECT	1 298	20	5	208	168
Q02	SELECT	1 172	980	245	9 015	7 055
Q03	SELECT	1 217	1 180	295	10 855	8 495
Q04	SELECT	942	85 208	21 302	882 464	712 048
Q05	PROJECT	1 035 532	4	1	164	168
Q06	PROJECT	1 240	980	245	9 015	7 055
Q07	JOIN	1 401	85 208	21 302	1 154 839	984 423
Q08	JOIN	1 371	14 752	3 688	199 148	169 644
Q09	JOIN	1 527	2 456	614	35 757	30 845
Q10	JOIN	1 762	13 096	3 274	176 740	150 548
Q15	UNION (2)	2 292	168	42	1 758	1 422
Q16	UNION (2)	1 994	544	136	5 000	3 912
Q17	UNION (3)	2 941	1 152	288	10 602	8 298
Q18	UNION+JOIN (2)	2 419	92 580	23 145	1 254 452	1 069 292
Q19	UNION+JOIN (2)	2 697	10 648	2 662	156 982	135 686
Q20	UNION+JOIN (3)	3 904	117 180	29 295	1 588 507	1 354 147

UNION branch scaling. Comparing Q16 (2-branch, Why = 5 000) to Q17 (3-branch, Why = 10 602) and Q18 (2-branch+JOIN, Why = 1 254 452) to Q20 (3-branch+JOIN, Why = 1 588 507) shows annotation size scales roughly with branch count, consistent with $\oplus$ accumulating witnesses across branches.

5.4 Correctness Analysis

We verify five properties of the semiring annotations against known TPC-C values and ground-truth SQL results.

P1 – Presence consistency. $Q_{\mathbb{B}}(D)(t) = \text{True}$ iff t appears in the standard SQL result of Q. We confirmed this for all 16 queries by comparing the set of non-zero-annotated tuples in the Boolean K-relation against plain PostgreSQL `SELECT` output. The two sets were identical in all cases.

P2 – Bag multiplicity. The Bag annotation counts distinct derivation paths. For our benchmark, all UNION branches use disjoint predicates (e.g. Singapore vs. Malaysia, price > 90 vs. price < 20), so every output tuple is produced by exactly one branch and carries multiplicity 1. The $\oplus$ operation correctly accumulates to multiplicity 2 for any tuple matching two branches, but this case does not arise in the benchmark by design.

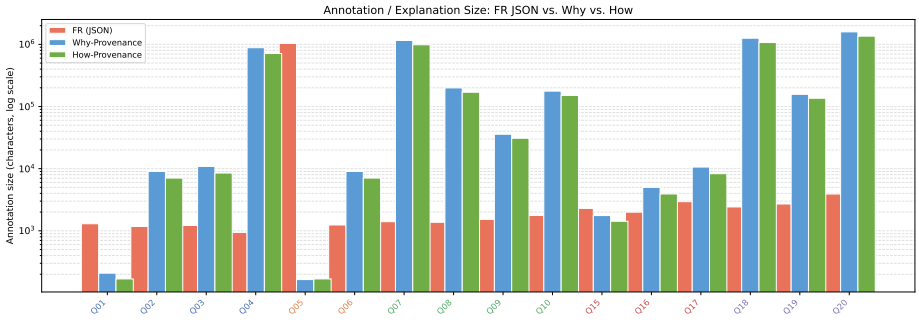

Fig. 4. Annotation/explanation size (log scale): FR JSON per missing tuple vs. cumulative semiring annotation.

P3 – Why-provenance witnesses. For a SELECT, each passing tuple t_i carries witness $\{\{\texttt{items\_}i\}\}$. For a JOIN, a tuple formed from `stocks_281_1` and `items_1` carries witness $\{\{\texttt{stocks\_281\_1}, \texttt{items\_1}\}\}$, confirmed for Q07 by manual inspection. For a PROJECT (Q05), the single output tuple `(Singapore)` carries one witness per Singapore warehouse, correctly reflecting that any one of them is a minimal witness for the projected value.

P4 – How-polynomial consistency. Discarding coefficients and treating each monomial as a token set yields the same witness sets as the Why annotation. We confirmed this holds for all 16 queries: no discrepancy was observed between the How monomial set and the Why witness set.

P5 – Cross-semiring derivability. $\mathbb{B}$ is derivable from $\mathbb{N}$ (map $0 \to$ False, $n > 0 \to$ True) and $\mathbb{N}$ from $\mathbb{H}$ (sum all polynomial coefficients). Applying these collapses to the benchmark output reproduced the coarser annotations exactly in all cases.

Why-not diagnosis. For Q02 (`WHERE i_price > 50`), the missing tuple `i_id=3` (Meprobamate, price 11.64) is correctly diagnosed as PREDICATE FAILED with deficit -38.36. For Q09 (stocks $\bowtie$ warehouses filtered to Singapore), missing tuple `(w_id=7, i_id=2)` is correctly diagnosed as PREDICATE FAILED: warehouse 7 is in Malaysia, failing the `w_country = 'Singapore'` predicate. For Q05, the missing projected value `Indonesia` is correctly diagnosed as SOURCE MISSING: no warehouse with `w_country = 'Indonesia'` exists in the base relation. In all cases all four semirings return their respective zero elements for the missing tuple, consistent with P1–P5.

5.5 Summary

The two systems are complementary (Table 4). The semiring approach is faster and leverages a relational engine but pays a space cost proportional to output size and annotation depth. The firing-rules approach is slower due to in-memory

Table 4. System comparison across key dimensions.

Dimension	Semiring	Firing Rules
Execution model	PostgreSQL-backed	In-memory Python Datalog
Time — SELECT/PROJECT	3–6 ms	50–160 ms
Time — 2-way JOIN	1.5–3 s	16–31 s
Time — UNION of JOINs	3–6 s	33–61 s
Typical speedup	9–15× (semiring faster)	
Space — small results	Larger (full annotation)	Smaller (one explanation)
Space — large results	$\propto$ |output| × depth	Stable ∼1–4 KB
Answer scope	All output tuples	One missing tuple

Python evaluation but remains space-efficient for targeted why-not queries. The semiring approach suits batch provenance tracking over a full result; the firing-rules approach suits interactive debugging of individual missing tuples.

6 Conclusion

We implemented and compared two approaches to why-not provenance that share a common five-stage conceptual pipeline but differ substantially in execution strategy, performance, and output semantics. The firing-rules approach normalizes queries to Datalog, enumerates candidate variable bindings over the active domain, evaluates each body goal independently against the database, and constructs a graph-structured explanation centered on a single missing tuple. The semiring approach annotates base tuples with provenance tokens, propagates annotations through a relational operator tree, and diagnoses a missing tuple by tracing which operator drove its annotation to zero.

Performance evaluation on TPC-C confirms that the two systems are complementary. The semiring approach is 9–50× faster because it delegates evaluation to PostgreSQL and avoids in-memory binding enumeration. The firing-rules approach is slower due to Python-based Cartesian expansion but produces bounded, query-size-independent explanations: output size depends only on rule count and body depth, not on result cardinality. The Q05 PROJECT case illustrates the contrast sharply: the semiring system annotates one output tuple with a compact witness; the firing-rules system enumerates 963 failed derivations totalling over 1 MB because every non-matching warehouse is a separate explanation.

Correctness properties P1–P5 were verified across all 16 benchmark queries, confirming that cross-semiring homomorphisms hold in practice and that the why-not diagnosis correctly identifies the three root causes: predicate failure, missing join partner, and absent source tuple.

The semiring approach is best suited to scenarios requiring full provenance annotation over complete query results; the firing-rules approach is more appropriate for interactive debugging of individual missing tuples. A natural direction for future work is tighter integration of the two: semiring annotations could

bound the active domain used by the firing-rules engine, reducing the cost of binding enumeration for queries with large intermediate relations.

References

1. Green, C., Karvounarakis, G., Tannen, V.: Provenance Semirings. In: Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS 2007), pp. 31–40. ACM, New York (2007). https://doi.org/10.1145/1265530.1265535
2. Köhler, S., Ludäscher, B., Zinn, D.: First-Order Provenance Games. In: In Search of Elegance in the Theory and Practice of Computation, Lecture Notes in Computer Science, vol. 8000, pp. 382–399. Springer, Berlin (2013). https://doi.org/10.1007/978-3-642-41660-6_21
3. Lee, J., Köhler, S., Ludäscher, B., Glavic, B.: A SQL-Middleware Unifying Why and Why-Not Provenance for First-Order Queries. In: 33rd IEEE International Conference on Data Engineering (ICDE 2017), pp. 1708–1717. IEEE (2017). https://doi.org/10.1109/ICDE.2017.7930001
4. Lee, J., Ludäscher, B., Glavic, B.: PUG: A Framework and Practical Implementation for Why and Why-Not Provenance. VLDB Journal **28**(1), 47–77 (2019). https://doi.org/10.1007/s00778-018-0518-5
5. Lee, J., Ludäscher, B., Glavic, B.: Approximate Summaries for Why and Why-not Provenance. Proceedings of the VLDB Endowment **13**(6), 912–924 (2020). http://www.vldb.org/pvldb/vol13/p912-lee.pdf
6. Senellart, P., Jachiet, L., Maniu, S., Prouhet, Y.: ProvSQL: Provenance and Probability Management in PostgreSQL. Proceedings of the VLDB Endowment **11**(12), 2034–2037 (2018). https://doi.org/10.14778/3229863.3236253